Laboratory report of Nanjing University of Information Science and Technology

Experiment Title Cloud Computing
Data 2026/4/27 Class AI 1
Name Weijie Xu ID 202383910029

一、Experimental purpose

(1) To understand and practice controlled traffic shifting, this experiment enables you to learn how to gradually shift traffic between different versions of a microservice in a Kubernetes service mesh using Istio.
(2) To master dynamic request routing techniques, you will practice applying Istio routing rules to direct incoming service requests to different versions of a microservice based on criteria such as version labels or custom request headers.
(3) To evaluate system resiliency via fault injection, this part of the lab teaches you to intentionally inject faults like delays and aborts into service calls at the application layer. The purpose is to assess the robustness of microservices and test how the application logic and Istio's failure recovery mechanisms handle situations where upstream services are slow or respond with errors.
(4) To implement and assess circuit breaking for resilience, you will configure circuit breaker settings using Istio destination rules to limit connection pools and detect outlier failures. The goal is to observe how circuit breaking prevents cascading failures by quickly failing requests to unhealthy service instances.
(5) To gain hands-on experience with Istio's traffic management capabilities, you will work on implementing traffic shifting strategies in a live environment. This involves directing specific subsets of requests to new service versions while monitoring performance and behavior.
(6) To test and fine-tune Istio's request routing capabilities, you will set up routing configurations to route traffic based on various criteria such as geographic location or request headers. This task demonstrates the flexibility of Istio's routing features, enabling the creation of dynamic, policy-driven routing solutions for complex service architectures.
(7) To assess the overall resilience of your system, you will integrate fault injection and circuit breaking into your traffic management strategies. This will allow you to observe the system's response to failures and ensure that Istio's traffic management tools can handle real-world disruptions.

二、Experimental Contents

The contents of this experiment involve a series of tasks aimed at exploring Istio's traffic management capabilities in a microservices architecture. These tasks are

designed to provide practical experience in traffic shifting, request routing, fault injection, and circuit breaking, all critical components in managing the traffic and resilience of distributed systems. The experiment will be conducted step-by-step, with each step requiring the configuration and testing of different Istio features to control, monitor, and troubleshoot service-to-service communication.

1. Traffic Shifting

In this task, I will configure Istio's traffic shifting functionality to manage deployments of different versions of a microservice. By using weighted routing rules, I will gradually shift traffic from one version of a service to another, observing the effects and ensuring smooth migration while minimizing risk.

2. Request Routing

This part of the experiment focuses on setting up Istio's routing capabilities to control the flow of traffic based on specific conditions such as request headers or service versions. This will allow me to dynamically control the routing of requests within the service mesh, providing insights into the flexibility of Istio's traffic management features.

3. Fault Injection

I will practice injecting faults such as delays, aborts, or errors into the service communication to simulate real-world issues like network latency or failure. By configuring Istio's fault injection features, I will test the resilience of my microservices and ensure that the system can handle disruptions effectively.

4. Circuit Breaking

The Bookinfo sample application was deployed to simulate a typical microservices architecture. Multiple services such as productpage, reviews, ratings, and details were created, forming a multi-tier service structure. Pod and service statuses were monitored to ensure successful deployment and proper integration with Istio.

5. Observability and Metrics Collection

I will integrate Istio's observability tools to monitor traffic, service performance, and health within the service mesh. By configuring Istio's telemetry features, I will collect metrics on request counts, latencies, and errors. This will enable me to visualize the impact of traffic management strategies on service performance and make data-driven decisions for further optimization.

三、Detailed steps of the experiment

The Kubernetes command-line environment has been checked here, confirming that kubectl is available before any Istio resources are installed or configured in the cluster.

```
● ubuntu@DESKTOP-CCHACFV:~/cloud_lab/gitea-lab$ kubectl version --client
  Client Version: v1.34.1
  Kustomize Version: v5.7.1
```

Istio has been successfully downloaded and extracted from the official repository, and the required control-plane and sidecar images are being pulled locally to support the later service mesh deployment.

```
ubuntu@DESKTOP-CCHACFV:~/cloud_lab/gitea-lab$ docker pull docker.io/istio/pilot:1.29.2
docker pull docker.io/istio/proxyv2:1.29.2
1.29.2: Pulling from istio/pilot
689b91d88a0f: Pull complete
dec211a11132: Pull complete
ede6d5e95b67: Pull complete
8c7170db3ec9: Pull complete
Digest: sha256:3681fe197870abad5c953107e241e104a813ba2d6e63933ff53e0545348ff8b1
Status: Downloaded newer image for istio/pilot:1.29.2
docker.io/istio/pilot:1.29.2
1.29.2: Pulling from istio/proxyv2
689b91d88a0f: Already exists
dec211a11132: Already exists
ede6d5e95b67: Already exists
2d099c62ee84: Pull complete
f354efe9a7ad: Pull complete
e65b0115582d: Pull complete
Digest: sha256:69ca4585993f4057861954f18be88df8268b713d114e7036bf9b185458c7a2c1
Status: Downloaded newer image for istio/proxyv2:1.29.2
docker.io/istio/proxyv2:1.29.2
```

The required Istio images have been loaded into the local Kubernetes cluster, which ensures that the cluster can start Istio components without repeatedly downloading images during the experiment.

```
ubuntu@DESKTOP-CCHACFV:~/cloud_lab/gitea-lab/lab04/istio-1.29.2$ kind load docker-image docker.io/istio/
pilot:1.29.2 --name lab04
kind load docker-image docker.io/istio/proxyv2:1.29.2 --name lab04
Image: "docker.io/istio/pilot:1.29.2" with ID "sha256:86f0d9b34281a823272d92a65fa26074b1d7ebdbe8ffcaa609
5c9ebf65ee048c" not yet present on node "lab04-control-plane", loading...
Image: "docker.io/istio/proxyv2:1.29.2" with ID "sha256:c99272fbb1fc7a39091da7aefeff63d16dfdb42e204acf8e
23a69dc1a75790d2" not yet present on node "lab04-control-plane", loading...
```

The Istio control-plane pod has been restarted and verified in the istio-system namespace, showing that the core service mesh component is running normally after the image and environment preparation.

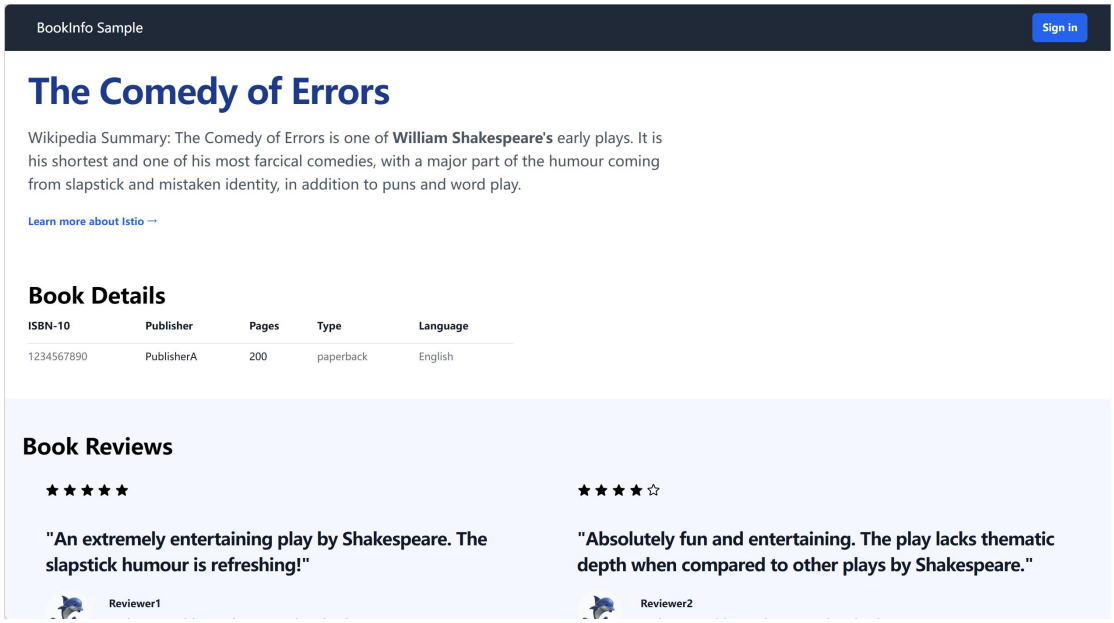
```
ubuntu@DESKTOP-CCHACFV:~/cloud_lab/gitea-lab/lab04/istio-1.29.2$ kubectl delete pod -n istio-system -l a
pp=istiod
kubectl get pods -n istio-system -w
pod "istiod-6766655967-qclwq" deleted from istio-system namespace
NAME                      READY   STATUS              RESTARTS   AGE
istiod-6766655967-27bhf   0/1     ContainerCreating   0          1s
istiod-6766655967-27bhf   0/1     Running             0          2s
istiod-6766655967-27bhf   1/1     Running             0          2s
```

The default namespace has been labeled for automatic Istio sidecar injection, so new application pods created in this namespace can join the service mesh and be managed by Envoy proxies.

```
ubuntu@DESKTOP-CCHACFV:~/cloud_lab/gitea-lab/lab04/istio-1.29.2$ kubectl label namespace default istio-i
njection=enabled
namespace/default labeled
ubuntu@DESKTOP-CCHACFV:~/cloud_lab/gitea-lab/lab04/istio-1.29.2$ kubectl get namespace default --show-la
bels
NAME      STATUS   AGE   LABELS
default   Active   63m   istio-injection=enabled,kubernetes.io/metadata.name=default
```

The Bookinfo application is now reachable through the browser, and the product page

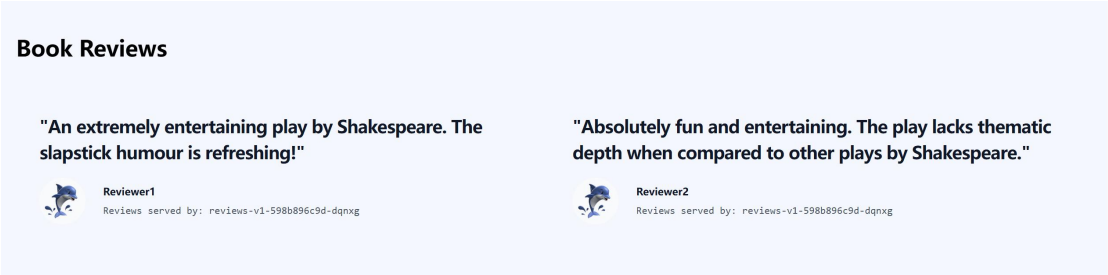
displays book details and review information, confirming that the sample microservices application has been deployed and exposed successfully.



The initial VirtualService configuration has been applied to route all Bookinfo service traffic to version v1, providing a stable baseline before testing more specific request-routing rules.

```
ubuntu@DESKTOP-CCHACFV:~/cloud_lab/gitea-lab/lab04/istio-1.29.2$ kubectl apply -f samples/bookinfo
/networking/virtual-service-all-v1.yaml
virtualservice.networking.istio.io/productpage created
virtualservice.networking.istio.io/reviews created
virtualservice.networking.istio.io/ratings created
virtualservice.networking.istio.io/details created
```

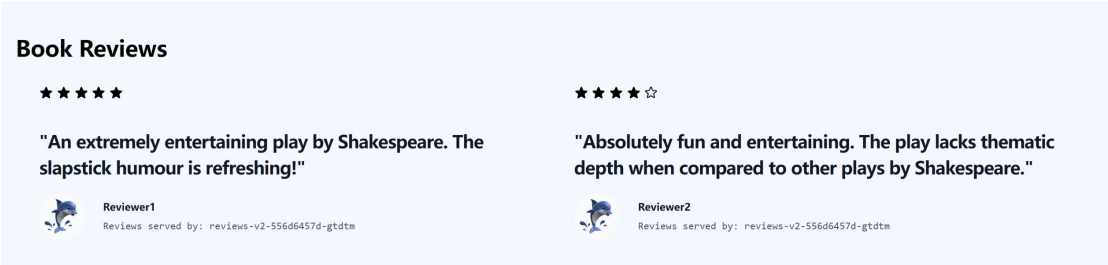
The review section is displayed without star ratings, which matches the expected behavior after directing reviews traffic to version v1.



The user-based routing rule has been applied for the reviews service, allowing requests from the specified test user to be directed to a different service version.

```
ubuntu@DESKTOP-CCHACFV:~/cloud_lab/gitea-lab/lab04/istio-1.29.2$ kubectl apply -f samples/bookinfo
/networking/virtual-service-reviews-test-v2.yaml
virtualservice.networking.istio.io/reviews unchanged
```

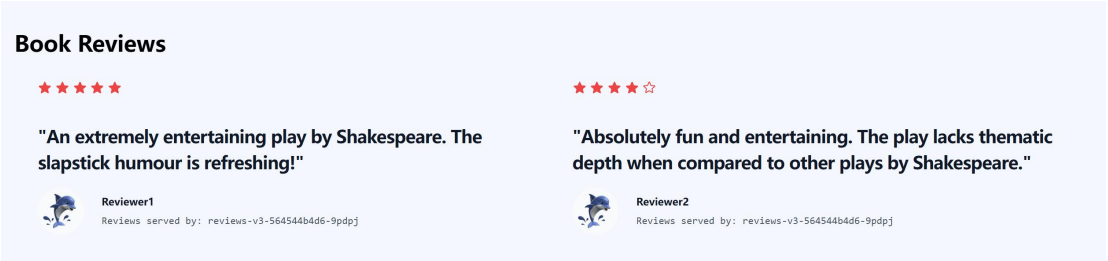
The review section now shows black star ratings after the routing rule takes effect, indicating that the request has reached reviews version v2 under the configured user condition.



The traffic-shifting rules have been updated for the reviews service, first splitting traffic between versions and then moving traffic toward version v3.

```
ubuntu@DESKTOP-CCHACFV:~/cloud_lab/gitea-lab/lab04/istio-1.29.2$ kubectl apply -f samples/bookinfo
/networking/virtual-service-all-v1.yaml
virtualservice.networking.istio.io/productpage unchanged
virtualservice.networking.istio.io/reviews configured
virtualservice.networking.istio.io/ratings unchanged
virtualservice.networking.istio.io/details unchanged
ubuntu@DESKTOP-CCHACFV:~/cloud_lab/gitea-lab/lab04/istio-1.29.2$ kubectl apply -f samples/bookinfo
/networking/virtual-service-reviews-50-v3.yaml
virtualservice.networking.istio.io/reviews configured
ubuntu@DESKTOP-CCHACFV:~/cloud_lab/gitea-lab/lab04/istio-1.29.2$ kubectl apply -f samples/bookinfo
/networking/virtual-service-reviews-v3.yaml
virtualservice.networking.istio.io/reviews configured
```

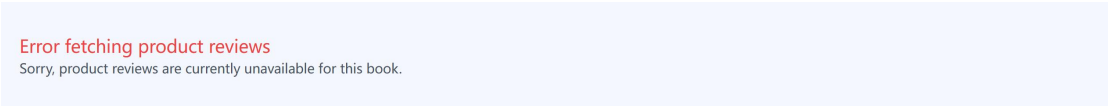
The review section shows red star ratings, demonstrating that traffic has been shifted to reviews version v3 and the new service version is being served.



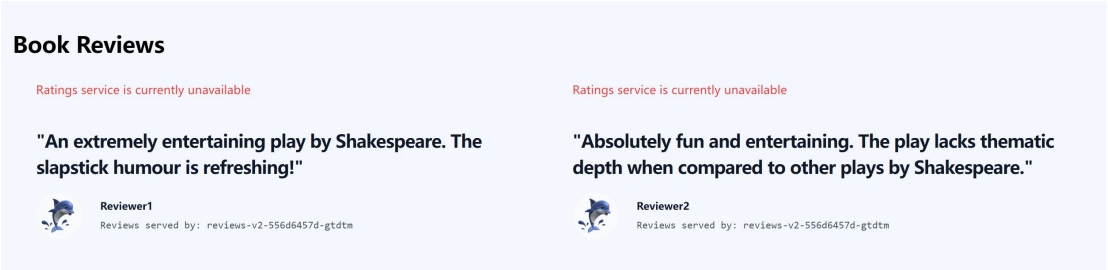
The fault-injection configuration has been applied to the ratings service, introducing controlled delay and failure behavior for the selected request path.

```
ubuntu@DESKTOP-CCHACFV:~/cloud_lab/gitea-lab/lab04/istio-1.29.2$ kubectl apply -f samples/bookinfo
/networking/virtual-service-all-v1.yaml
kubectl apply -f samples/bookinfo/networking/virtual-service-reviews-test-v2.yaml
virtualservice.networking.istio.io/productpage unchanged
virtualservice.networking.istio.io/reviews configured
virtualservice.networking.istio.io/ratings unchanged
virtualservice.networking.istio.io/details unchanged
virtualservice.networking.istio.io/reviews configured
ubuntu@DESKTOP-CCHACFV:~/cloud_lab/gitea-lab/lab04/istio-1.29.2$ kubectl apply -f samples/bookinfo
/networking/virtual-service-ratings-test-delay.yaml
virtualservice.networking.istio.io/ratings configured
```

The product page reports an error when fetching reviews, showing how the application behaves when the downstream ratings service is affected by injected faults.



The final routing adjustment restores the reviews service configuration after the fault-injection test, preparing the application for continued validation or cleanup.



四、Reflection

1. Through this experiment, I gained valuable hands-on experience in managing traffic and ensuring the resilience of a microservices architecture using Istio. By implementing traffic shifting, I learned how to smoothly transition between different versions of services, which is essential for canary deployments and reducing risks in production environments. The request routing task helped me understand how to direct traffic based on specific conditions, such as headers or version labels, which is crucial for enabling more flexible and dynamic application behaviors. Additionally, fault injection allowed me to simulate real-world failures, providing insights into how Istio’s failure recovery mechanisms work and how they can be leveraged to improve system resilience.

2. Configuring circuit breaking was another key takeaway, as it taught me how to prevent cascading failures by quickly failing requests to unhealthy service instances. This helps maintain the stability of the system, especially during heavy load or network disruptions. Furthermore, I explore Istio’s observability tools, which provided a deeper understanding of traffic patterns and system health through real-time metrics.

3. The load balancing configurations demonstrated how Istio efficiently distributes traffic across multiple service instances, ensuring high availability and fault tolerance. Overall, this experiment gave me a comprehensive understanding of how Istio can be used to manage, secure, and optimize traffic in a microservices architecture.

4. This experiment also highlighted the importance of observability in a microservices environment. By integrating Istio's telemetry features, I gained firsthand experience in monitoring the traffic flow and understanding service performance through metrics such as request count, latency, and error rates.

5. Additionally, configuring security with Istio helped me appreciate the critical role of service-to-service authentication and encryption in modern architectures. This step emphasized the growing need for robust security measures in cloud-native

environments, where services often interact over shared networks.

附录 程序命令代码

附录 1：工具检查
kubectl version --client kubectl cluster-info istioctl version

附录 2：Istio 安装、命名空间注入
istioctl install --set profile=demo -y kubectl label namespace default istio-injection=enabled kubectl get pods -n istio-system kubectl get svc -n istio-system kubectl describe pod -n istio-system -l app=istiod kubectl describe pod -n default -l app=details

附录 3：Bookinfo 基础部署
kubectl apply -f samples/bookinfo/platform/kube/bookinfo.yaml kubectl get pods kubectl get services kubectl get pods -w kubectl apply -f samples/bookinfo/networking/destination-rule-all.yaml kubectl get destinationrules

```
kubectl get destinationrules -o yaml

kubectl apply -f samples/bookinfo/networking/bookinfo-gateway.yaml

kubectl get gateway

kubectl get virtualservice

kubectl get svc -n istio-system
```

附录 4：WSL、kind 场景访问方式
kubectl port-forward svc/istio-ingressgateway -n istio-system 8080:80 浏览器访问：http://localhost:8080/productpage

附录 5：Request Routing 请求路由
kubectl apply -f samples/bookinfo/networking/virtual-service-all-v1.yaml kubectl get virtualservices -o yaml kubectl get destinationrules -o yaml kubectl apply -f samples/bookinfo/networking/virtual-service-reviews-test-v2.yaml kubectl get virtualservice reviews -o yaml kubectl apply -f samples/bookinfo/networking/virtual-service-reviews-50-v3.yaml kubectl get virtualservice reviews -o yaml kubectl apply -f samples/bookinfo/networking/virtual-service-reviews-v3.yaml

```
kubectl get virtualservice reviews -o yaml
```

附录 6：Fault Injection 故障注入

```
kubectl apply -f samples/bookinfo/networking/virtual-service-all-v1.yaml
kubectl apply -f
samples/bookinfo/networking/virtual-service-reviews-test-v2.yaml
kubectl get virtualservice reviews -o yaml
kubectl apply -f
samples/bookinfo/networking/virtual-service-ratings-test-delay.yaml
kubectl get virtualservice ratings -o yaml
kubectl apply -f
samples/bookinfo/networking/virtual-service-ratings-test-abort.yaml
kubectl get virtualservice ratings -o yaml
```

附录 7：Circuit Breaking 熔断

```
kubectl apply -f samples/httpbin/httpbin.yaml
kubectl get pods -l app=httpbin
kubectl get svc httpbin
kubectl apply -f samples/httpbin/sample-client/fortio-deploy.yaml
kubectl get pods -l app=fortio
export FORTIO_POD=$(kubectl get pods -l app=fortio -o
'jsonpath={.items[0].metadata.name}')
```

```
kubectl exec "$FORTIO_POD" -c fortio -- /usr/bin/fortio curl -quiet
http://httpbin:8000/get
kubectl exec "$FORTIO_POD" -c fortio -- /usr/bin/fortio load -c 2 -qps 0
-n 20 -loglevel Warning http://httpbin:8000/get
kubectl exec "$FORTIO_POD" -c fortio -- /usr/bin/fortio load -c 3 -qps 0
-n 30 -loglevel Warning http://httpbin:8000/get
kubectl exec "$FORTIO_POD" -c istio-proxy -- pilot-agent request GET
stats | grep httpbin | grep pending
```